Bachelor of Science in Software Engineering
June 2022

Differences in performance between containerization & virtualization

With a focus on HTTP requests

Johannes Berggren
Jens Karlsson

Faculty of Computing, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden

This thesis is submitted to the Faculty of Computing at Blekinge Institute of Technology in partial fulfilment of the requirements for the degree of Bachelor of Science in Software Engineering. The thesis is equivalent to 10 weeks of full time studies.

The authors declare that they are the sole authors of this thesis and that they have not used any sources other than those listed in the bibliography and identified as references. They further declare that they have not submitted this thesis at any other institution to obtain a degree.

Contact Information:
Authors:
Johannes Berggren
E-mail: jobj16@student.bth.se

Jens Karlsson
E-mail: jeks19@student.bth.se

University advisor:
Michel Nass
Department of Software Engineering

Faculty of Computing
Blekinge Institute of Technology
SE–371 79 Karlskrona, Sweden

Internet : www.bth.se
Phone : +46 455 38 50 00
Fax : +46 455 38 50 57

Abstract

Containerization and virtualization are two of the keystones of cloud computing. Neither technologies are a new invention but did not become widely used until it regained popularity through new implementations. Virtualization regained popularity with the founding of VMWare, and containerization has become vastly popular in the last decade with Docker. When using a service from a Cloud Service Provider today, that service will more than likely be utilizing one of these technologies. This study aims to compare the performance of these two technologies when being used to host an API and how they utilize their provided hardware resources to handle HTTP requests. A series of load tests were conducted on an API developed and hosted on the two technologies to measure the hardware performance, response time and throughput of each technology. Hyper-V was used for virtualization, and Docker was used for containerization. Data was collected on resource utilization, response time, and throughput. The data was also compared to related research to validate it. The results of the experiment showed that, in our implementation, virtualization was superior to containerization in every measured aspect. We conclude that containerization has a bottleneck in the implementation we chose that impedes the container's network performance, which results in the container not being able to process as many HTTP requests as the virtualized environment. The number of processed HTTP requests for the container in relation to CPU usage is superior to that of the virtualized environment, which leads us to believe that it could be possible that the container would be superior if not for the network performance.

Keywords: Containerization, Virtualization, Response time, Throughput

Contents

Chapter 1

Introduction

1.1 Background

Virtualization is a technology that creates a abstracted layer over computer hardware that allows the hardware of a single computer to be utilized and divided into multiple virtual computers, known as virtual machines. Each virtual machine runs its own operating system. Virtual machines could be used if you want different or multiple of the same operating systems on the same physical computer. *Containerization* is a technology that allows software code to be packaged into containers that only requires the host operating systems libraries and dependencies to be able to function. Containerization could be used for hosting micro services in an isolated environment, such as a REST API.

Virtualization is a technology that has been increasing rapidly with digitalization. As P. Vinit describes it, "Digitalization is a fundamental disruptive force triggered by Fourth Industrial Revolution and Internet of Things, which has changed the way we approach and think about business processes and activities." [45]. Technologies like Internet of things (IoT), the different types of cloud services like Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS) has helped companies develop and deploy products at a faster pace [49] [39] [27]. From a survey conducted by Balakrishna Narasimhan and Ryan Nichols [44] in 2010, they found that 68% said they would have most of their applications and platforms in the public cloud within three years(2013). Container-based virtualization, also known as containerization, has become very popular since the introduction of Docker [4] and it is a lightweight alternative to more traditional hypervisor-based virtualization. Docker is one of the most popular software tools available and is still growing fast according to their index [15].

Virtualization and the introduction of data centers have had positive effects on companies regarding the financial aspect by limiting the requirement of purchasing hardware for hosting their products and maintaining the infrastructure of hosting their servers in-house. N. Krishnadas and P. Radhakrishna [43] found that the smaller companies, by their definition below 100 servers, could save more than 200% of those companies current IT infrastructure budget by implementing cloud computing instead of hosting their own server infrastructure. The implementation of virtualization also has an effect on the environment [40] [47] [32] with more companies using cloud services for their products [44], which in practice makes more companies share hardware between each other if they are hosted on the same data center, as the US National Institute of Standards and Technology(NIST) defines

cloud computing; "Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction. This cloud model is composed of five essential characteristics, three service models, and four deployment models.".

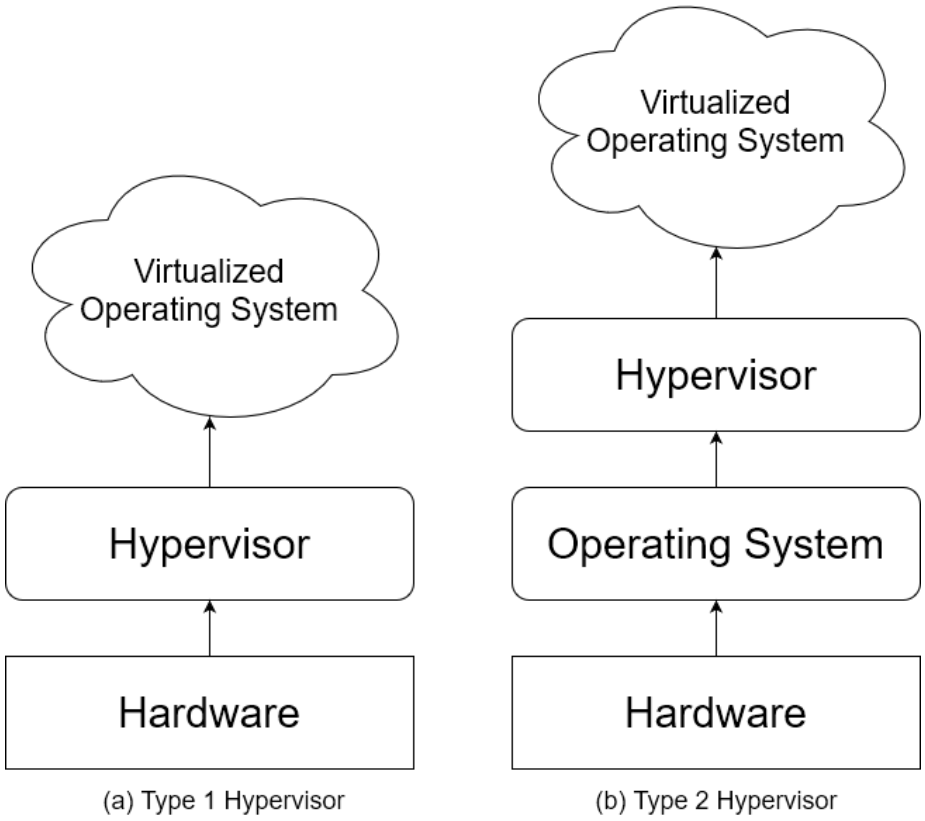

Figure 1.1: Architecture of hypervisor type 1 & hypervisor type 2

Both technologies share similarities. Hypervisor type 1 is the hypervisor used for our experiment, also known as Bare-metal hypervisor [13]. As shown in 1.1, Type 1 hypervisor hosts its virtual machine on top of the hardware and not on top of the host operating system as hypervisor type 2. This gives type 1 hypervisors the ability to utilize hardware more directly and efficiently than type 2 hypervisors that utilize the hardware simulated by the host operating system.
Containerization's architecture 1.2 may appear similar to type 2 hypervisors. The significant difference between them is that a container engine can have many containers hosted through the host OS kernel, while a type 2 hypervisor needs to create a complete guest OS for each virtual machine that needs to be created.

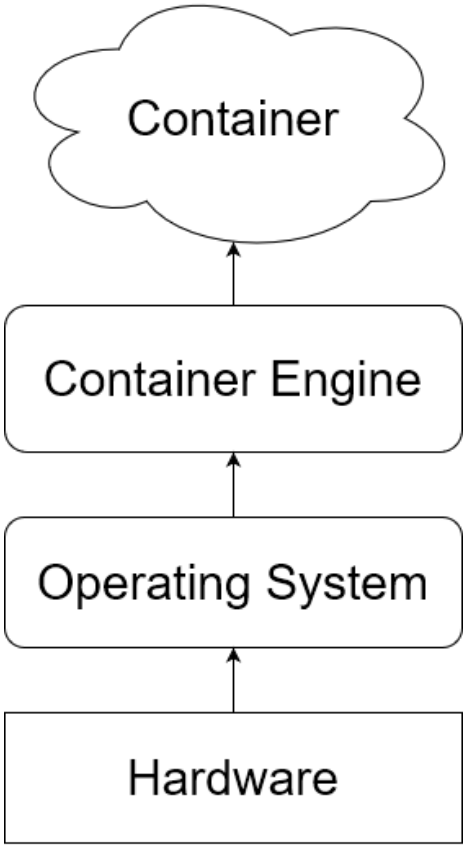

Figure 1.2: Architecture of containerization

1.2 Scope

The scope of this research is to measure and compare the performance differences in CPU- and physical memory performance, response time, and throughput, between containerization and virtualization. It will be measured solely on the handling of HTTP requests themselves, meaning no additional overhead such as inserting or retrieving data from a database, calculations, or large amounts of return data will be included. The experiment will be carried out with virtual hardware limitations on the two technologies to simulate a low tier machine provisioned by a cloud service provider.

1.3 Outline

With these technologies sharing many similarities, the question arises whether or not one is objectively better than the other or if they are just meant to be used for different purposes. In essence, that is what this research aims to create a better understanding of, particularly in regards to APIs and processing HTTP requests.

This research looks closer into the performance differences between virtualization and containerization regarding hosting an API on a web server. It has an experimental part in which the two technologies are put through identical tests, with the results of these tests being processed and analyzed. The data will also be compared against previously available knowledge to control the validity of the results.

Chapter 2

Literature Review

In this section we will review research on virtualization, containerization, performance comparisons between virtualization and containerization, cloud computing and benefits of using cloud services.

2.1 Virtualization

Virtualization is the technology of creating a virtual computer on a abstract layer above the host computer's hardware. A. Hwang et al. [35] describes virtualization with "Virtualization technology provides a way to share computing resources among VMs by using hardware/software partitioning, emulation, time-sharing, and dynamic resource sharing. Traditionally, the operating system (OS) controls the hardware resources, but virtualization technology adds a new layer between the OS and hardware.". We visualize the architecture of virtualization in 1.1, showing both types of virtualization, Hypervisor type 1 and Type 2.

2.1.1 Hypervisor type 1 vs. Hypervisor type 2

We touched upon the differences between hypervisors in 1.1, as we mentioned they are different in their architecture. Hypervisor type 1 hosts virtual machines on top of the hardware and in that way is more efficient in it's way of utilizing the available hardware. Hypervisor type 2 hosts virtual machines on top of the hosts operating system, giving it a bigger overhead before reaching the hardware. Z. Li et al. [37] compares type 2 hypervisors to containerization and state that a hypervisor type 2 that requires access to the physical hardware through it's host operating system needs binaries and libraries for it's guest operating system and therefor will create resource competition between the applications running on the VM and the guest operating system.

2.1.2 Hyper-V

Hyper-V is a Microsoft developed type 1 hypervisor that is included in certain x64 versions of Windows operating systems. *Hyper-V* is a micro-kernelized hypervisor, which is a architecture that does not require any drivers for the physical hardware to be installed in the hypervisor, instead they are using a parent partition that holds all drivers for the physical hardware, see figure 3.1 for a simple visualization of the architecture. H. Fayyad-Kazan et al. [34] raises an advantage of micro-kernelized

hypervisors that is that device drivers do not need to be hypervisor aware which gives micro-kernelized hypervisors a wider range of supported devices. They also raise a disadvantage of micro-kernelized hypervisors that they require an operating system in the parent partition before the hypervisor can operate.

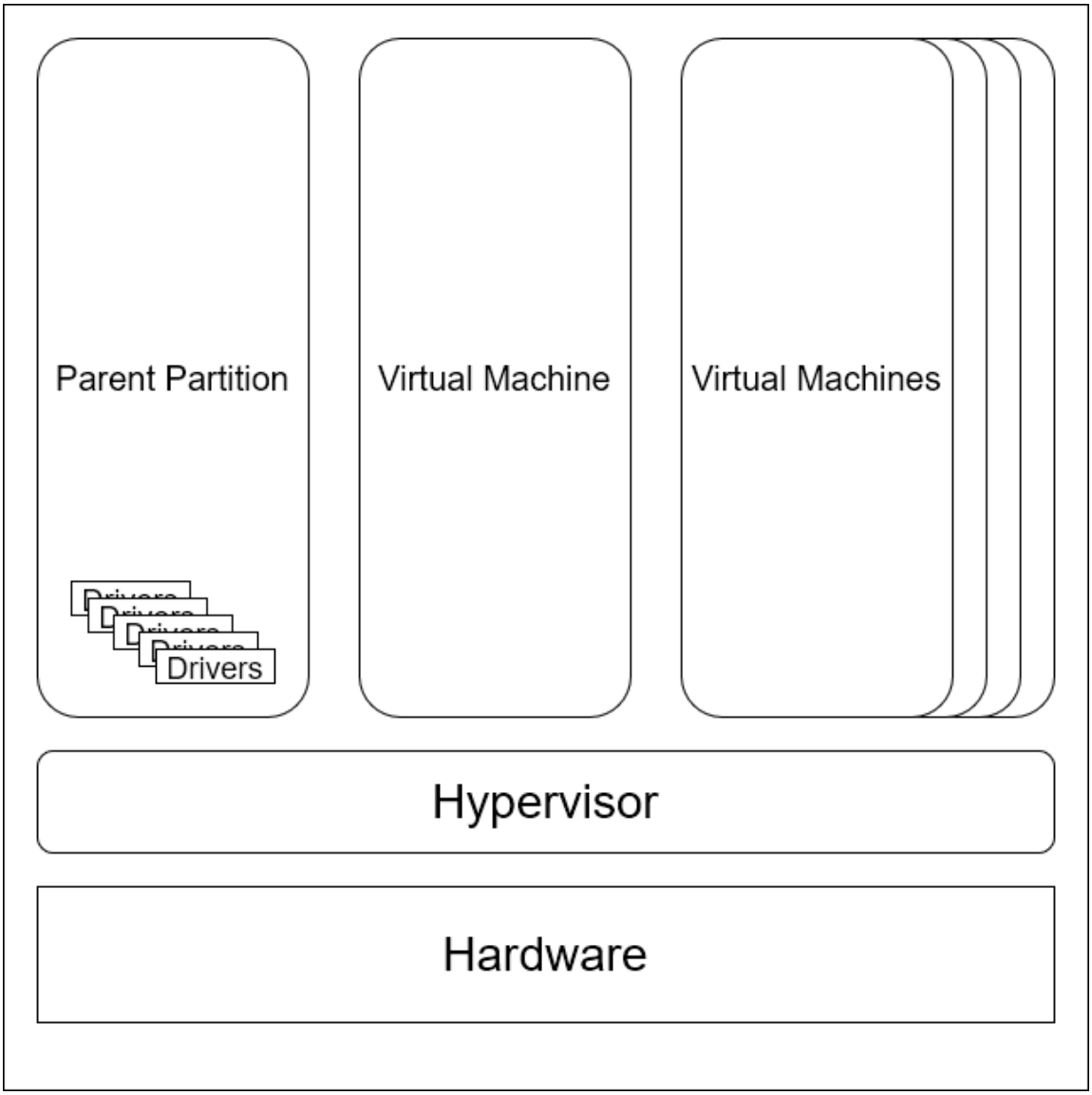

Figure 2.1: Simplified architecture of Micro-kernelized hypervisor [20]

2.1.2.1 Hyper-V compared against other Type 1 hypervisors

Comparisons between different hypervisors are an extensively researched area, and many vary in which part of the virtual machines they test. To support Hyper-V as a valid choice to represent virtualization we reviewed previous studies on Type 1 hypervisor performance comparisons.
A. Elsayed & N. Abdelbaki [33] compared VMware ESXi5, Citrix Xen and Hyper-V. They found that Hyper-V has the worst average CPU utilization of the three

compared hypervisors. Comparing Order per minute (OPM) showed that Hyper-V had the highest of the three. They also found that VMWare ESXi5 and Hyper-V were evenly matched in memory utilization while Citrix Xen was the worst in this scenario.
Hwang et al. conclude their comparison by comparing four hypervisors: Hyper-V, KVM, vSphere, and Xen. "We believe that the results of our study demonstrate the benefits of building highly heterogeneous data center and cloud environments that support a variety of virtualization and hardware platforms." [35].

2.2 Containerization

Containerization is a technology that utilizes Linux cgroups and namespaces primarily to create isolated, lightweight environments to run specific applications and code. These containers are tailor-made for their respective applications with only the required libraries and dependencies. Preeth et al. [42] describe the separation of containers as "Multiple containers can share the same kernel, but each container can be constrained to only use a defined amount of resources available in the host machine." The kernel, in this case, refers to the kernel of the host operating system. This is visualized in figure 1.2.

2.2.1 Docker vs. Other containerization technologies

Even though Docker has become almost synonymous with containerization, it is not the only software that utilizes this technology. Another reasonably widespread tool for containerization is Linux Containers (LXC) [9]. The two technologies are very similar in CPU and network performance, according to a study done by Ákos Kovács [36]. Another study done by Harri Manninen et al. [41] shows much the same when it comes to the CPU performance, although Docker has a minor edge on LXC. It is worth mentioning that Docker does utilize LXC, but they are still different softwares that could be used for containerization independently of each other.

2.2.2 Popularity of Docker

Docker has had and is continuing to have an explosion in popularity during the past couple of years, there has been a year-over-year increase in new Docker Hub accounts of approximately 45% and a year-over-year increase of Docker Hub pulls of 145% [14]. *Docker Hub* is Docker's repository of images used to build one's own Docker containers. This has resulted in the name Docker becoming almost synonymous with containerization.

2.3 Virtualization vs. Containerization

While virtualization and containerization are similar technologies that are often confused for one another, they do have some substantial differences.

2.3.1 Differences in technology

Preeth et al. [42] describe some differences between Virtual Machines and Docker containers. One of these differences is the operating system. When creating a VM, one must choose an operating system to install. On the other hand, a container uses the kernel of one's current operating system and adds libraries and dependencies from the chosen image. This results in VMs taking up a great deal more storage space than containers do. Another difference Preeth et al. bring up is overhead. Docker is lightweight with a low complexity compared to the more complex Virtual machine, which results in less overhead.

2.3.2 Pros and cons

A significant advantage of containerization is the speed at which one can get an application up and running. A container requires no OS installation or creation of users within the system. This is a stark difference from a VM, where one has to install and configure an entire OS before even being able to run their application. R.K Barik et al. [29] bring up an argument for the extra configuration since that same configuration that is required on the VM makes it much easier to manage system policies, network, users, and security.

2.4 Cloud computing

Cloud computing is a practice where containerization and virtualization are heavily implemented in the different types of services cloud services providers offer. In this section, we will go through some of the usages of virtualization and containerization in cloud computing, looking at different softwares that have been developed and different products that companies have that utilize either or both of the technologies.

2.4.1 Containerization in cloud computing

We conjecture that implementing one single container is not commonly done for usage in cloud computing, and we have therefore chosen to look at a container orchestration system, such as Kubernetes [22] that utilizes the strength of containerization's lightweight implementation and scalability.
VG da Silva et al. writes [31] "Kubernetes dominates the adoption on the scheduling and orchestration layers; it has monolithic scheduling properties. Docker Swarm is an offering from the Docker container platform. It is no match for Kubernetes in terms of ease of use and functionality.". They also mention that Google, Microsoft, VMWare, IBM, and Red hat supported Kubernetes since it launched, since then more big cloud computing companies have endorsed Kubernetes [21] [18] [17] [23] [26] [16] [25] [19] [24].
D. Bernstein writes in an article [30]"Those who want to deploy applications with the least infrastructure will choose the simple container-to-OS approach. This is why container-based cloud vendors can claim improved performance when compared to hypervisor-based clouds."

2.4.2 Virtualization in cloud computing

A. Randal states [46] that the origins of both virtual machines and containers could be traced to the end of the 1950s. A. Randal also states that the interest in virtual machines and containers started to gain more traction in the late 1990s with the introduction of Disco and later VMware.
RK Barik et al. [29] highlights the importance of virtualization in cloud computing in their analysis, "Virtualization is a crucial part in the definition of Cloud computing. Resources that are virtualized in the cloud then can be managed more efficiently. Virtualized Cloud Infrastructure provides the abstraction required to make sure that an application or business service model is independent of the underlying hardware such as servers, storage or networks.".

2.4.3 Cloud Data centers

Comparisons between virtualization and containerization are the main objective of this thesis, but implementing these technologies is an exciting topic that we will touch upon briefly.

2.4.3.1 Growth of cloud data centers

The growth of global communication is vast. Y. Liu et al. [40] state that it was estimated that 97% of the global network traffic would be related to data centers in 2020. W Van Heddeghem et al. [48] find a trend that between 2005 and 2010, the growth of volume servers by 5.9% p. a., an increase of 13.1% p. a. of high-end servers and the mid-range servers decreased by 5.3% p. a.

2.4.3.2 Need for sustainable data centers

With the growth of IoT, social media, the various entertainment streaming services, and other industries that utilize the cloud, the demand for cloud data centers (CDC) is also increasing each year, which increases the global energy consumption of CDCs. J. Shuja et al. [47] state that the estimated global energy consumption of CDCs was 2.4% and that they would grow annually by 15-20% and that the CDCs were responsible for 2% of global CO_2 emissions.
W Van Heddeghem et al. [48] found that the Compound Annual Growth Rate of data centers between 2007 to 2012 was 4.4% and that cooling and power supply losses dominate it. Only about 40% of the power consumption is used for powering the servers.
Y. Liu et al. [40] centralized scenario of data centers being created around the Pan-Arctic region showed a 720 million ton reduction of CO_2 compared to a decentralized scenario, showing the importance of where data centers are being located.
A positive trend is that leading cloud service provider companies are setting goals of 100% green renewable energy in the future. Some have already reached goals of being carbon neutral (i.e., for every kWh of electricity consumed, they purchase a KWh of renewable energy) [1] [6] [7] [11] [12]. Hopefully this trend for sustainability will continue, and that companies will reach their goals since the need for CDCs most likely will increase with increased automation and digitalization in society.

Chapter 3

Method

Hypervisor-based and container-based virtualization play significant roles in cloud services, but their usage varies greatly. If a user wants a small micro-service for sporadic use, their choice would most likely be a container-based solution. If it is something more permanent that does not need scaling or any significant changes, the user would most likely go with a hypervisor-based solution.

Nevertheless, if the user is a small business owner and does not see the need for a cloud service and their only need for their business is a simple website, which one would be best?

3.1 Objective

These research questions will compare virtualization and containerization and create more knowledge about the differences between these two technologies. The objective is to show if one of the technologies is more efficient in handling HTTP requests and resource utilization than the other.

3.2 Research questions and Motivations

In this thesis, we aim to share more knowledge on the following questions.

- RQ1: What are the differences in throughput between containerization and virtualization?

  The throughput will give us input if one of the technologies is more proficient in their way of handling requests for our implementation. *Throughput* is a measurement that calculates number of requests through total time, the formula is: Throughput = (number of requests) / (total time). The time is calculated from the first sample to the end of the last sample [3].

- RQ2: What are the differences in response time between containerization and virtualization?

  Response time is the time it takes from when a client sends a request to the reception of the response. This question will answer if one of the technologies is faster than the other in returning requests for our implementation.

- RQ3: What are the differences in resource utilization between containerization and virtualization?

  Differences in resource utilization in the different technologies have in performance regarding their CPU usage and physical memory usage. The results will provide knowledge on if one of the technologies is using their provided hardware more efficiently than the other and if that may show a relation between their results and the other results in our measurements.

3.2.1 Approach

Our approach to answering these questions is to conduct an experiment toward a containerization and a virtualization environment. We will also compare our findings to other research found during our literature review .

For the experiment on both technologies, an Apache server was created that hosted a Django REST API that would respond with a 200 HTTP status code if reached successfully. If a request were unsuccessful, it would respond with an error code. An error was triggered if the request took to long to be handled by the server. Both technologies were limited to the same amount of CPU cores and physical memory to provide an equal comparison. In that way, the focus of the experiment would be how they use these provided resources.

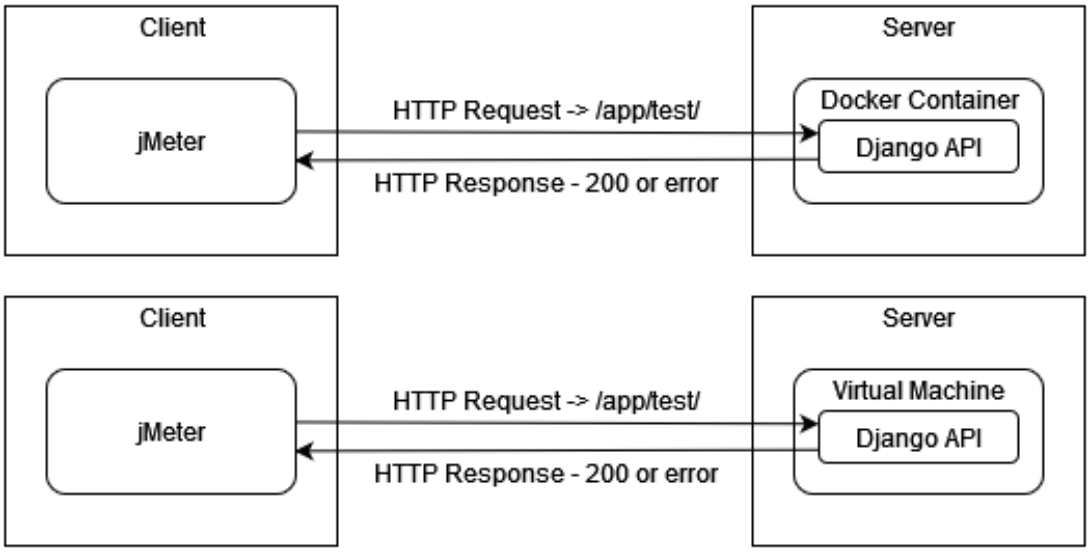

Figure 3.1: Test setups for both containerization(top) and virtualization(bottom).

For the containerization technology Docker [4] was used. A Dockerfile A.1 based on the Ubuntu 18.04 image was created with all needed dependencies. For virtualization technology, Hyper-V [10] was used that then hosted a Ubuntu server 18.04 virtual machine.

Docker was chosen due to its popularity in the industry and convenience based on previous knowledge from working with Docker. Before installing Docker, WSL2

first had to be installed as it is used for the Docker backend. The Linux OS installed on WSL2 was Ubuntu 18.04. When installing Docker, we made sure to specify that WSL2 was to be used instead of HyperV.

Hyper-V was chosen for virtualization. *Hyper-V* is a type 1 hypervisor also known as bare metal hypervisor [13]. Our experiments were conducted on a computer with a freshly installed Windows 10 Host OS, to ensure no previously installed software affected the tests.

3.2.2 Hardware specifications

- Processor: Intel i9 9900k 3.6GHz 8 Cores
- Physical Memory: 32gigabyte DDR4-2666 MHz
- GPU: Nvidia GeForce RTX 2080
- Motherboard: Asus Z390 ATX GL12CX

3.2.3 Data collection

- jMeter on the client-side to send HTTP requests to the API and analyze the responses from these requests.
- Sysstat to monitor and record the resource utilization of the environments.

The environments were limited to two CPU cores and four gigabytes of physical memory. This setup was chosen based on the "bottom" tier virtual machine setups from the three current leading [28] cloud service providers, Google Compute Engine, AWS (Amazon Web Services), and Microsoft Azure. This configuration is not the lowest they go, AWS, for instance, has a lower tier than this, but it is the lowest setup they have in common. To limit the Docker containers hardware, the options "–cpus=2" and "memory=4GB" were used when the container was started. The limitations of the VM were put in place using the HyperV settings.

The decision to go for the lowest tier possible was made because this research, as mentioned in the introduction 1, is geared towards creating more knowledge on the subject for smaller companies who might not want to or need to make substantial investments in cloud services or on-premise infrastructure. A downside to limiting the hardware this low is that the knowledge created might not be applicable to implementations deployed on more powerful hardware. It could be possible that such an implementation would behave differently enough that the results of a similar experiment would be different.

3.3 Experimental approach

Before comparing both technologies, a load test was conducted to see where the limit was for both technologies. If a server responded with an error code, the test would fail, and the limit for that environment would be known. We repeated the tests five times to see if it was an arbitrary failure.

For testing, we decided to test the environments with a step scaling approach. Tests were conducted on both technologies on 20, 40, 60, 80, 90, 95, and 100% of the max load. We chose to have smaller steps when approaching the max load with the thought that, the closer to the max load, the more potential errors or major differences would occur between the two technologies.

When conducting our load test we choice an arbitrary number of 10 000 for the amount of HTTP requests sent per user. The max load test reached its first error code at 13 users each sending 10 000 requests. This resulted in the max load of 130 000 HTTP requests divided by 13 users. Each test was repeated ten times, and the results are the mean of these repeated tests.

The tests were conducted on the same computer for hosting both the Docker container and the virtual machine. The tests were conducted on a small network, and the computer was connected to the router using an Ethernet cable. The client-side that sent the HTTP requests were connected to the same network as the host through WiFi. This approach gave us the knowledge that both technologies had the same starting point regarding hardware available for utilization. This approach ensured that both tests had access to the exact same hardware, as opposed to if we would have used separate computers to host the Docker container and the virtual machine. A different approach that would have been closer to a realistic scenario would have been to have multiple clients send requests instead of one client using threading to mimic multiple clients. The resources for such an approach were not available for this thesis.

The scope of our approach is also narrowed to the handling of an HTTP request. Experiments on a broader architecture would have been interesting and could generate a different result. However, this would have been too extensive of an experiment to include in this research.

3.3.1 Result Compilation

3.3.1.1 Jmeter

Jmeter has an option to get the results compiled into an HTML report [2] when running the test. The values taken from the reports were then put in a spreadsheet that calculated the total average of all variables from the completed tests for that bracket.

3.3.1.2 Sysstat

Each reading from sysstat measured CPU usage and memory usage every second for the entire duration of the test. A Bash script was created to calculate the average of each sub-test then a total average was calculated from all sub-tests. Another calculation had to be added for the docker readings, as the sysstat readings included the whole system's capacity and not just the container. The formula used to calculate the usage:

$$\frac{percentageUsed}{(assignedCores/totalCores)} \times 100 \tag{3.1}$$

3.4 Motivation of choices

The approach to choosing software and technologies for this thesis was dependent on if there was any previous experience or knowledge of the required choices. Some choices were made out of convenience, others out of previous knowledge, and some out of popularity while searching for software or technology.

3.4.1 Containerization & Virtualization technologies

Docker was chosen based on both its popularity and convenience from previous projects. Hyper-V was chosen out of convenience because our school provided us with a free Windows Educational license and Hyper-V is included in that license [8]. Hwang et al. [35] compares four type 1 hypervisors and concludes that they all have different strengths and weaknesses, and your choice of hypervisor should vary depending on its intended area of use. To conduct our experiments on a type 1 hypervisor, we made a choice to have an operating system that had as little overhead as possible. As we mentioned in section 1, a type 2 hypervisor has the disadvantage of being hosted on top of the host operating system suffering from the overhead of that operating system, while a type 1 hypervisor is hosted directly on top of the hardware of the host computer and is free to utilize all the hardware directly without having to go through a host operating system 1.1 .

3.4.2 Software for network testing

We had no previous knowledge of testing network applications. When choosing a software to perform the tests, we searched for free open software that had a good reputation, and Jmeter is one that was frequently suggested by multiple blogs and websites.

3.4.3 Server side performance testing

For measuring the performance of the technologies in regards to the utilization of the provided hardware, we used sysstat. Sysstat is a built-in software on Ubuntu, and since both the virtualization and containerization technologies were created on Ubuntu systems, we chose sysstat out of convenience,

3.4.4 Server side Application and Deployment

Django is a web framework written in Python. It was chosen for the server-side application since we had extensive previous experience using the technology, specifically the Django REST framework module, from previous projects and employment. It also has an easy-to-use package called mod_wsgi, which generates a ready-to-use instance of Apache2 for running the service in production mode.

Our method will allow us to provide more knowledge in comparisons of type 1 hypervisors and containerization and, therefore, not for all types of virtualization. It

will also only provide specific knowledge in the two softwares we have chosen to represent each technology. However, since Docker is one of the most popular containerization softwares [15] our choice for Docker as the representative of containerization was a simple one. For virtualization, our choice was made out of convenience, and as we have previously stated, there are different strengths and weaknesses of all type 1 hypervisor software.

Chapter 4

Results and Analysis

In this chapter, the results of the conducted experiments will be presented and analyzed, as well as our answers to the research questions stated in the Method chapter 3.

4.1 Experimental results

The results in the tables below were all collected on the same system described in the Method chapter, and apart from the 100% load tests, all were done after each other. The 100% load tests were done separately due to time restraints. The result of the 100% test differs from the other tests, this is most likely caused by it being conducted at another time and that both the client and server computers have been rebooted from the other tests.
A variety of different data points have been collected during the course of the tests.

- Number of requests is the total number of requests sent from the test client to the server per test.
- Amount of error codes is the total amount of error codes generated during each test.
- The response time minimum is the lowest recorded time for any one request to give a response during a test. This is **not** a compounded number, but the individual lowest time, measured in milliseconds.
- The response time maximum is the highest recorded time for any one request to give a response during a test. This is **not** a compounded number, but the individual highest time, measured in milliseconds.
- Response time average is an average of all response times recorded during one test, measured in milliseconds.
- Throughput is the number of successful requests completed per second.
- CPU usage is the amount of total assigned CPU that is being utilized, measured in percent.
- Memory usage is the amount of total assigned Memory (RAM) that is being utilized, measured in percent.

Docker							
Measurement	20% Load	40% Load	60% Load	80% Load	90% Load	95% Load	100% Load
Number of requests	26000	52000	78000	104000	117000	123500	130000
Amount of error codes	0	4	0	8	5	22	13
Response time minimum*	2	2	1.9	1.8	1.9	1.9	1.9
Response time maximum*	300	146.3	241.5	550.2	762.1	639.1	107.5
Response time average*	8.16	8.15	8.21	8.42	8.76	8.32	8.23
Throughput**	1503.21	1540.11	1538.77	1505.69	1460.23	1526.93	1544.80
CPU usage	22.63	24.89	25.46	25.73	24.85	25.93	24.91
Memory usage	10.07	14.3	14.97	15.14	15.26	15.24	14.16

Table 4.1: Test results for containerization

Hyper-V							
Measurement	20% Load	40% Load	60% Load	80% Load	90% Load	95% Load	100 % Load
Number of requests	26000	52000	78000	104000	117000	123500	130000
Amount of error codes	0	0	0	0	0	0	0
Response time minimum*	1.2	1	1	1	1	1	1
Response time maximum*	269.3	185.1	361.5	170.6	188.3	167.8	81.4
Response time average*	5.03	4.97	4.96	4.96	4.93	4.88	4.50
Throughput**	2343.21	2474.63	2509.61	2528.97	2552.31	2575.84	2793.85
CPU usage	44.49	50.81	55.77	55.35	56.02	57.02	52.91
Memory usage	18.58	18.61	18.64	18.68	18.71	18.74	18.66

Table 4.2: Tests results for virtualization

*Response time is represented in milliseconds

**Throughput is amount of HTTP requests and is represented in requests per seconds

The choice of limiting the CPU and physical memory did not provide any limitations for either technology as neither of them used all their available resources.

4.1.1 RQ1: What are the differences in throughput between containerization and virtualization?

As shown in Figure 4.6, there is a substantial difference in the throughput of the two technologies. The VM had an average of 65% higher throughput, but it also got consistently better as the load increased. The container's throughput did not improve in a measurable way between 20% and 100% load. The VM, on the other hand, improved steadily between the different loads. The VM's lowest recorded throughput was even higher than the containers' highest recorded throughput.

> RQ1 Summary: Virtualization was always superior to containerization in regards to throughput in the experiment. The VM had a steady increase in throughput between each test, whereas the Docker container's throughput did not increase in a significant way between any of the tests.

4.1.2 RQ2: What are the differences in response time between containerization and virtualization?

The response time differences have three components: the shortest response time, the longest response time, and the average response time. As can be seen in figures 4.3, 4.4 and 4.5, the VM is superior to the Docker container in all three categories.

The shortest and longest response times are not representative of the response times as a whole, as they are just the edge cases of the tests, which is why the average response time is the measurement we consider to have the most weight.

4.1.2.1 Shortest Response Time

The shortest response time differences between Docker and the VM were similar. While there was an increase in time of almost 100% between the VM and Docker 4.3, the actual time difference was around 1ms.

4.1.2.2 Longest Response Time

However, the longest response time had a significant difference 4.4. It, too, had a high percentage difference at around 97%, but the actual time difference was around 200ms, which is a much more noticeable difference than the previous 1ms.

4.1.2.3 Average Response Time

The average response time aligns pretty well with the throughput. The average response time for Docker was around 66% longer than that of the VM 4.5, this corresponds with a time difference of around 3.34ms.

> RQ2 Summary: The response time difference between the two technologies is in favour of virtualization in all of the measured categories with it being more than two times faster than Docker on average.

4.1.3 RQ3: What are the differences in resource utilization between containerization and virtualization?

The resource utilization has two parts, CPU utilization and Memory utilization. The choice of limiting the CPU and physical memory, as we show in our result tables 4.1 4.2, did not provide any limitations for either technology as neither of them used all their available resources. From the data collected on the memory utilization during our tests, which can be seen in figure 4.2, we have concluded that the memory usage had no major impact on the tests due to the values showing no significant change. CPU usage, on the other hand, had some exciting results. The data shows that the VM can use twice the CPU assigned compared to the Docker container. However, it is essential to mention that for our implementation the Docker container can utilize the CPU it uses more efficiently.

4.1.3.1 CPU Utilization

The CPU utilization of the Docker container and the VM 4.1 follows similar curves. If we look at them individually, Docker uses the resources more efficiently, but the VM can use more of the CPU that it has been assigned. The VM, however, can utilize more of the CPU. Docker only uses an average of around 25% at its highest point, while the VM uses around 53%. This results in a difference in resource utilization of

around 110%. It is not linearly reflected in the differences in throughput (VM was around 65% better), but it is still a substantial difference.

4.1.3.2 Memory Utilization

There were minimal changes to the memory utilization through most of the tests. Except for one outlier, there were no significant changes in memory usage between the different tests. The tests for the VM had a total increase in memory usage of around 0.16%.

> RQ3 Summary: Memory utilization had minimal changes and has therefore been ruled out as relevant to the study. CPU utilization has advantages and disadvantages in both technologies. The VM can use over twice as much of its assigned CPU, but the Docker container can utilize its assigned CPU more effectively.

4.1.4 CPU

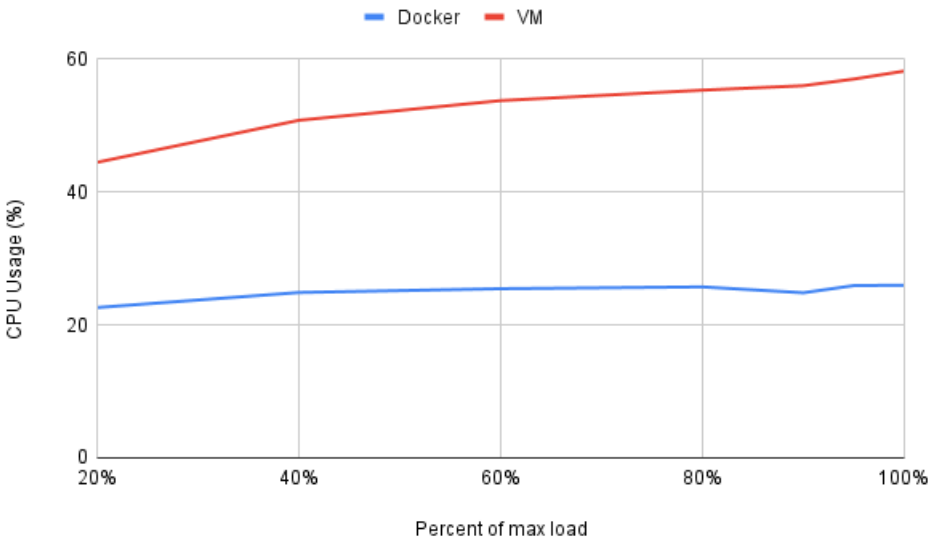

Figure 4.1: CPU Usage results

As can be seen in the figure above, the VM was vastly superior in utilizing the CPU. It used on average 110% more of the CPU than the Docker and had a more significant increase in CPU usage between the different test configurations. If one compares the CPU usage with the throughput (Figure 5.7), one can deduce that while the VM can use more of the CPUs provided, the Docker container gets a 30% higher throughput in relation to the amount of CPU it uses.

4.1.5 Memory

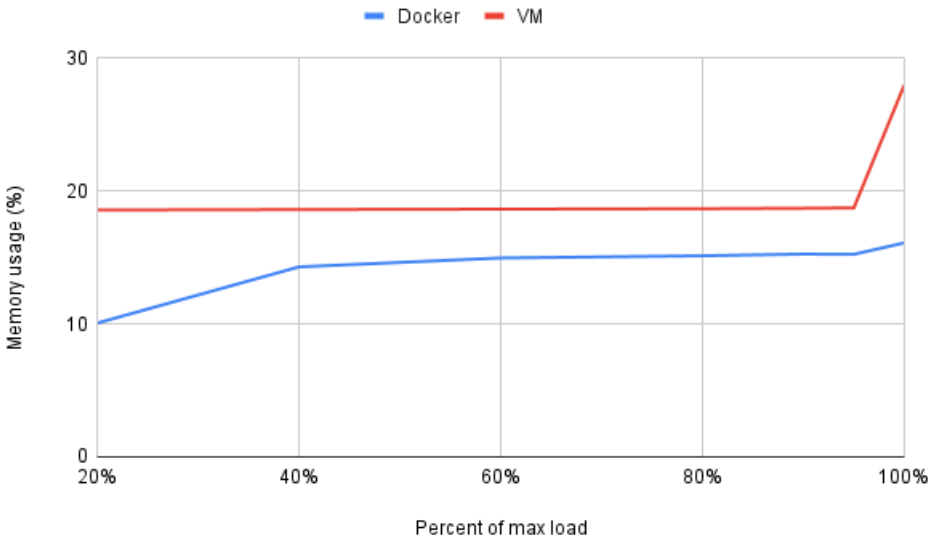

Figure 4.2: Memory Usage results

The memory usage does not appear to be affected by the software tests. As can be seen on the VM, the memory usage stays within fractions of a percent in all configurations except one. The outlier test in question was conducted at another time, which could be why it is so different. The Docker tests had a more considerable variation of memory usage of around 6%.

4.1.6 Number of Errors

While it is true that the Docker container had errors and the VM had none, we believe that the number of errors is so low that it should not be counted as a deciding factor, as the percentage of errors per test bracket are fractions of a percent as shown in table 4.1 and table 4.2.

4.1.7 Shortest Response Time

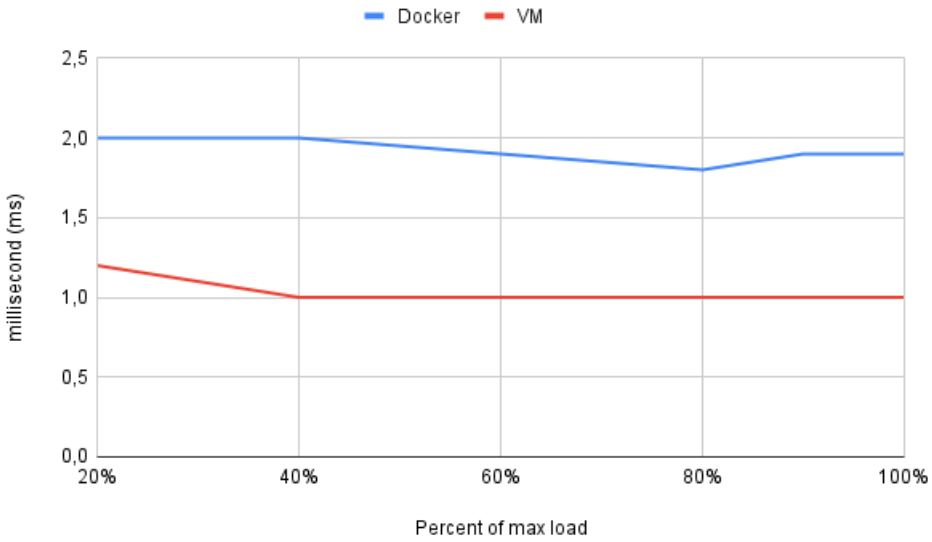

Figure 4.3: Shortest response times

The measured data shows that the shortest response time remains somewhat consistent throughout the test brackets for both technologies. It also shows that the VM is always around 100% faster than the Docker container.

4.1.8 Longest Response Time

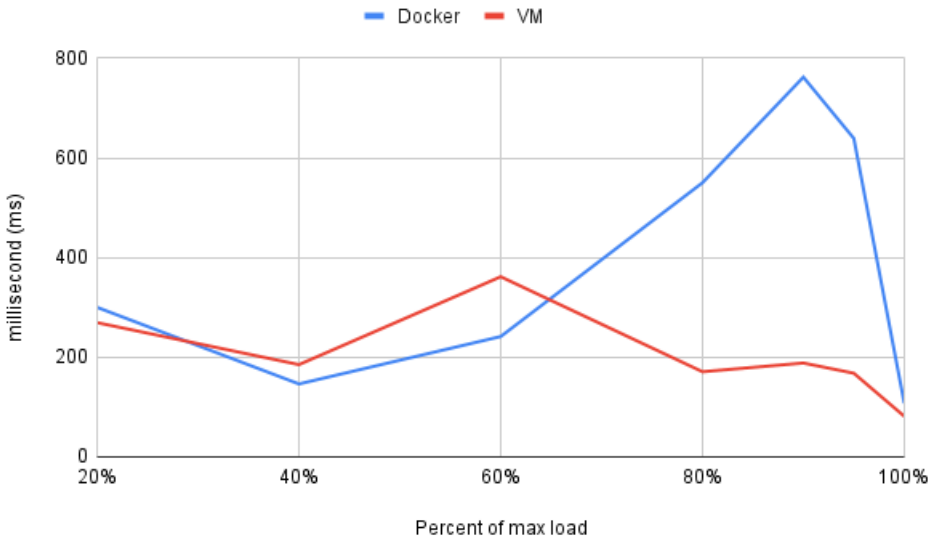

Figure 4.4: Longest response times

The longest response time remains somewhat similar between the two technologies at the start, up to the 60% bracket. After this, the longest response time of the Docker container starts to fall behind, reaching a high of near 400% of the VM at the 90% test bracket and then dropping down to almost the same value as the VM for the last test. While there is a clear difference between the technologies, this is only the single longest measured response time per test bracket and should not be considered representative of the elapsed response times as a whole.

4.1.9 Average Response Time

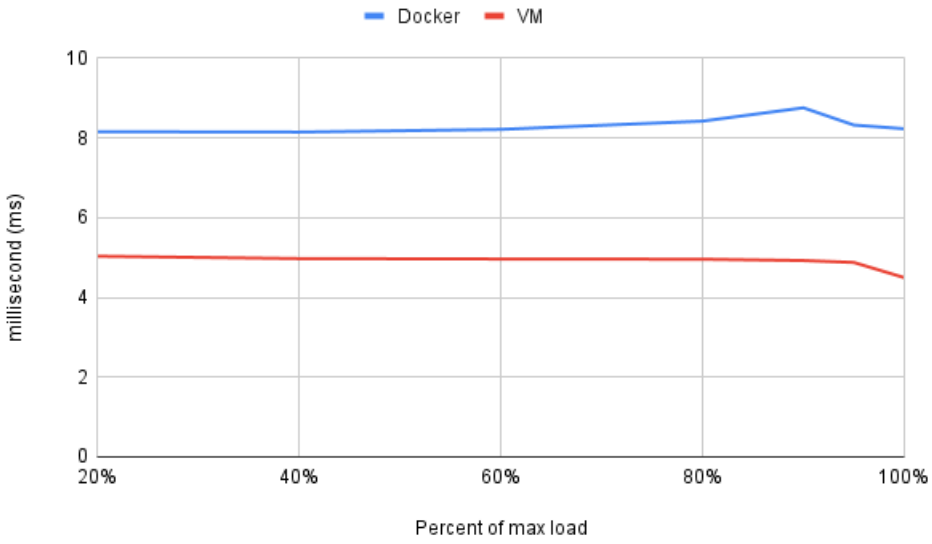

Figure 4.5: Average response times

The average response time for the two technologies stays roughly the same at all test brackets, with a slight deviation at 90% for Docker and 100% for the VM. As far as which one is better, the tests ran on the Docker container take an average of 70% longer than the tests on the VM.

4.1.10 Throughput

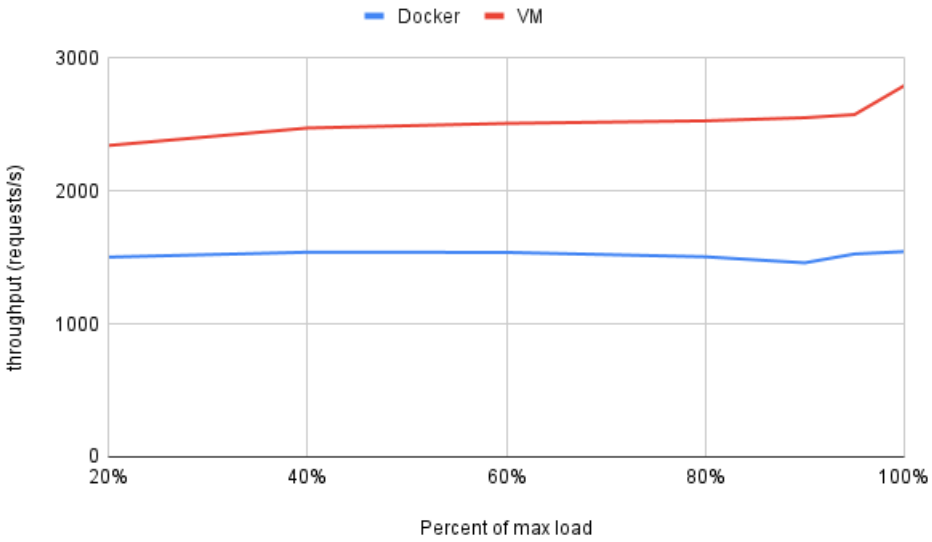

Figure 4.6: Throughput results

The throughput is a result of all the other measurements. Here, the VM has a significant advantage over the Docker container. The difference in the throughput was 67% in favor of the VM. The VM also improves as we move up the test brackets, which indicates that the throughput could go higher should the load increase. On the other hand, the Docker throughput remains at almost the same level throughout all the test brackets, indicating that the Docker container has reached its peak at the first bracket with a 20% load.

4.1.11 90-95% deviation

When we look at the deviation between 90 and 95%, we have a hypothesis that this is caused by the amount of long response time requests that this test interval has. If we compare it to figure 4.4 we see that the longest responses time is decreasing at the same interval. The downside of the graph in 4.4 is that it only shows the absolute longest response time. But there could be a correlation that the amount of long response times went down between these test cases. If we compare figure 4.4, 4.5 and 4.6 we can see that there is a correlation between them. If the amount of long response time goes down, the average response time also goes down, and throughput goes up. We do not know what caused this, there could be many factors but we would only be speculating in what actually happened.

Chapter 5

Discussion

We have come to a hypothesis regarding our results and related research. When looking at primarily the CPU utilization (figure 4.1), the average response time (figure 4.5), and throughput (figure 4.6), it is apparent that the graphs showing the Docker results have very similar curves. We believe that Docker has such a low CPU utilization compared to the VM because it is simply not getting enough requests to perform the way it should. This hypothesis is primarily based on two prior pieces of research. Z Li shows [38] that Docker is the superior choice when it comes to raw CPU performance when compared to Hyper-V. This partially matches our results because Docker could utilize the CPU used more effectively. However, it still had a lower overall CPU usage, which can be explained by jointly examining Li's research and that of Ákos Kovács. Kovács' research [36] shows that Docker has worse network performance than that of the host OS. As Hyper-V is a bare-metal hypervisor, the network performance should be the same as a host OS.

In conjunction with our findings, these two pieces of research show that Docker indeed has an advantage in CPU performance but falls short in network utilization. However, this does not mean that a VM would always be superior to a Docker solution when deploying an application that utilizes HTTP requests in some form. It would all come down to traffic and the actions performed by the server. If the application in question will have relatively low traffic but handle tasks requiring a lot of CPU performance, then a Docker solution could be the right way. On the other hand, if the application is expected to have very high traffic, but the CPU load is not particularly demanding, then a VM could be the better choice.

5.1 Implications for the Industry

As previously stated in the Introduction chapter 1, the point of this research is not to revolutionize the Cloud Computing Industry as a whole but rather to supply knowledge to new, small businesses that do not have any previous experience. This thesis could provide some information on the subject and help those who are unsure of which technology would suit their needs the best and, therefore, perhaps aid with the choice of cloud service.

5.2 Unexpected Findings

An aspect of these tests that we had not thought of when gathering the empirical data was the usability and accessibility of the two technologies. In our experience, setting up a VM is more time-consuming than setting up a Docker container. When setting up a VM, an operating system has to be chosen, go through the installation process, and set up the environment with the required dependencies before being able to deploy the application. When using Docker, there is a high probability that there is a ready-made image for the programming language or even a specific framework available using Docker's repository of images called Docker Hub [5]. If the found image is not enough, it can be used as a base to add further dependencies and modifications. However, this does present a downside with Docker in that it has a steeper learning curve. To set up a Docker image for the code, a Dockerfile has to be created. A Dockerfile is a set of instructions for building a Docker image. This is where the project code can be copied to the container, dependencies installed, any eventual ports exposed, and optionally bind the Docker file system to the host file system so that changes can be made while running the container. Installing an operating system on a VM may be time-consuming. However, for someone working in a position where this would occur, it is probably a pretty straightforward process, as opposed to writing a Dockerfile with no previous experience.

Chapter 6

Conclusion

In this thesis we have conducted a literature review of virtualization and containerization as well as a comparison of the two softwares to similar softwares with the same technology. We have also touched upon their growth, impact on development processes, and how they are used on a broader scale in cloud computing and cloud data centers. Lastly, we mentioned the impact cloud data centers have had and the rapid development within that field.

According to the conducted experiments, there are substantial differences in performance between virtualization and containerization. It is, however, not the case that one technology is superior to the other. Instead, they are similar technologies meant for different applications. In conjunction with related research, we have come to a theory that our Docker setup has a choke point at the network level, which results in the CPU performance appearing worse. However, it is not getting an opportunity to perform at total capacity. This could be seen as a win for the virtual machine as the focus is on applications utilizing HTTP requests, but our tests do not simulate "real" traffic; instead, they are load tests. Docker could be a good choice for CPU-intensive applications, especially with some form of orchestration solution. A virtual machine solution could also be valid if squeezing out the last bit of CPU performance is unnecessary.

Chapter 7

Validity

This section will cover the threats that we have identified to the validity of our research.

7.1 Internal validity

We refer to the threats that our research setting posed to the thesis by internal validity.

7.1.1 Wi-Fi

We were located in a public building with many devices that share the same WiFi frequency, which could cause issues with the network's performance. R. Xu et al. [50] state, "Sharing the same frequency band definitely leads to cross technology interference. It will cause intermittent network connectivity, packet loss and ultimately result in lower network throughput and higher communication latency."

7.1.2 Windows Subsystem for Linux(WSL2)

We hypothesize that WSL2 could cause a loss in performance for Docker, but we have also found blogs that state that it could impact performance, and we have found blogs that say that it does not. We have not been able to find any credible sources that support either statement. This needs to be validated by future research by implementing a Docker environment on a Linux OS to validate if WSL2 has an impact on performance for Docker.

7.1.3 Open network

We conducted our experiment on a lab network that we did not control who had access to, and this could have interfered with our network traffic and caused performance losses for both testing environments.

7.1.4 Time of experiment

The time we conducted our experiment could also have caused threats to our performance. Based on the previous sections of Open Network and WiFi, a test during

a regular workday could have more performance interference than one during the evening or a weekend.

7.1.5 Tests conducted at different occasions

We conducted our tests for 20, 40, 60, 80, 90, and 95% tests for both environments at the same occasion. The 100% tests for both environments were conducted at a different occasion, which appears to cause our measurements to have some differences in value. Since it is a drastic change from the previous test increment, this could cause a loss of credibility to our 100% tests.

7.1.6 Setup of experiment

An error in implementing one of the softwares used to set up the testing environment could impact the validity of our study. We do not have vast experience in any of the softwares that we have implemented and could have created an uncaught error.

7.1.7 Data collection software

Sysstat is a performance monitoring tool that we used in both environments. Sysstat is not able to isolate and monitor only the CPU usage of the Docker container; therefore, we had to do an equation, as we show in equation 3.1, to calculate the % of CPU the Docker container utilized. Any error or miscalculation in this equation could impact the validity of the results from the Docker environment.
JMeter was used both for sending the HTTP requests and also to monitor the response time, errors and throughput. To verify the results of our research, a different software could be used to generate the same test cases.

7.2 External validity

In this section, we will discuss threats caused to the validity of our thesis if it were to be implemented in an environment that is not confined to our specific implementation.

7.2.1 Realistic implementation

Our API implementation is specified to a simple 200 HTTP status response code on a successful connection. From our point of view, this implementation is not a realistic scenario for either technology. At a minimum, the API would most likely return a value from a database or something equivalent. This would create a more considerable impact on the server and generate more CPU load, possibly creating a different result than ours.

Chapter 8

Future Work

There are several different approaches one could take in this subject for future work. One implementation we would find interesting is a comparison of a Type 1 hypervisor as we have used compared to a container orchestration implementation where a cluster of shared containers has the same physical resources as the type 1 hypervisor has. This would highlight the strength of containerization by dividing smaller workloads into multiple containers instead of one single container as we had in our implementation.
Research into our hypothesis that a container is bottle-necked by the network bridge to the host OS is also interesting, at least on a Windows OS with WSL2 used to host the Docker engine.

Appendix A

Supplemental Information

```
FROM ubuntu:18.04

ENV PYTHONUNBUFFERED=1

WORKDIR /code

COPY requirements.txt /code/
COPY ./project/ /code/

RUN apt-get update && apt-get upgrade -y &&\
    apt-get install python3 -y &&\
    apt-get install python3-pip -y &&\
    apt-get install gcc -y &&\
    apt-get install make -y &&\
    apt-get install apache2 -y &&\
    apt-get install apache2-dev -y &&\
    apt-get install libexpat1-dev -y &&\
    apt-get install python3-dev -y &&\
    pip3 install -r requirements.txt &&\
    python3 manage.py migrate

EXPOSE 8000:8000

CMD python3 manage.py runmodwsgi --working-directory . --user www-data --group www-data
```

Figure A.1: Dockerfile

References

[1] Amazon sustainability. (Accessed: May 11, 2022). [Online]. Available: https://sustainability.aboutamazon.com/environment/the-cloud?energyType=true

[2] Apache jmeter generating report dashboard. Accessed: May 5, 2022. [Online]. Available: https://jmeter.apache.org/usermanual/generating-dashboard.html

[3] Apache jmeter glossary. (Accessed: 27.04.2022). [Online]. Available: https://jmeter.apache.org/usermanual/glossary.html

[4] Docker. (Accessed: April 14, 2022). [Online]. Available: https://www.docker.com

[5] Docker hub. (Accessed: May 5, 2022). [Online]. Available: https://hub.docker.com

[6] Google sustainability. (Accessed: May 11, 2022). [Online]. Available: https://blog.google/outreach-initiatives/sustainability/dear-earth/

[7] Ibm sustainability. (Accessed: May 5, 2022). [Online]. Available: https://newsroom.ibm.com/2021-02-16-IBM-Commits-To-Net-Zero-Greenhouse-Gas-Emissions-By-2030

[8] Install hyper-v on windows 10. Accessed: April 29, 2022. [Online]. Available: https://docs.microsoft.com/en-us/virtualization/hyper-v-on-windows/quick-start/enable-hyper-v

[9] Linux containers. (Accessed: May 11, 2022). [Online]. Available: https://linuxcontainers.org

[10] Microsoft hyper-v. (Accessed: April 14, 2022). [Online]. Available: https://docs.microsoft.com/en-us/windows-server/virtualization/hyper-v/hyper-v-technology-overview

[11] Microsoft sustainability. (Accessed: May 11, 2022). [Online]. Available: https://blogs.microsoft.com/blog/2021/07/14/microsoft-cloud-for-sustainability-empowering-organizations-on-their-path-to-net-zero/

[12] Salesforce sustainability. (Accessed: May 11, 2022). [Online]. Available: https://www.salesforce.com/news/press-releases/2021/09/21/salesforce-achieves-net-zero-across-its-full-value-chain/

[13] What is a bare metal hypervisor? (Accessed: April 14, 2022). [Online]. Available: https://www.vmware.com/topics/glossary/content/bare-metal-hypervisor.html

[14] (2021) Docker index shows continued massive developer adoption and activity to build and share apps with docker. (Accessed: May 5, 2022). [Online]. Available: https://www.docker.com/blog/docker-index-shows-continued-massive-developer-adoption-and-activity-to-build-and-share-app

[15] (2021) Docker index shows momentum in developer community activity. (Accessed: 29 April 2022). [Online]. Available: https://www.docker.com/blog/docker-index-shows-surging-momentum-in-developer-community-activity-again/

[16] (2022) Alibaba kubernetes. (Accessed: May 6, 2022). [Online]. Available: https://www.alibabacloud.com/product/kubernetes

[17] (2022) Aws eks. (Accessed: May 6, 2022). [Online]. Available: https://aws.amazon.com/eks/

[18] (2022) Azure kubernetes. (Accessed: May 6, 2022). [Online]. Available: https://azure.microsoft.com/en-us/services/kubernetes-service/

[19] (2022) Google cloud engine. (Accessed: May 6, 2022). [Online]. Available: https://cloud.google.com/kubernetes-engine

[20] (2022) Hyper-v architecture. (Accessed: 5 May 2022). [Online]. Available: https://docs.microsoft.com/en-us/virtualization/hyper-v-on-windows/reference/hyper-v-architecture

[21] (2022) Ibm kubernetes. (Accessed: May 6, 2022). [Online]. Available: https://www.ibm.com/cloud/kubernetes-service

[22] (2022) Kubernetes. (Accessed: 6 May 2022). [Online]. Available: https://kubernetes.io/

[23] (2022) Oracle kubernetes. (Accessed: May 6, 2022). [Online]. Available: https://www.oracle.com/cloud-native/container-engine-kubernetes/

[24] (2022) Redhat openshift. (Accessed: May 6, 2022). [Online]. Available: https://www.redhat.com/en/topics/containers/red-hat-openshift-kubernetes

[25] (2022) Tencent kubernetes. (Accessed: May 6, 2022). [Online]. Available: https://intl.cloud.tencent.com/products/tke

[26] (2022) Vmware kubernetes-grid. (Accessed: May 6, 2022). [Online]. Available: https://tanzu.vmware.com/kubernetes-grid

[27] M. Attaran and J. Woods, "Cloud computing technology: improving small business performance using the internet," *Journal of Small Business & Entrepreneurship*, vol. 31, no. 6, pp. 495–519, 2019, accessed: May 4, 2022. [Online]. Available: https://doi.org/10.1080/08276331.2018.1466850

[28] R. Bala, B. Gill, D. Smith, D. Wright, and K. Ji, "Magic quadrant for cloud infrastructure and platform services," 2021, accessed: April 14, 2022. [Online]. Available: https://www.gartner.com/doc/reprints?id=1-271OE4VR&ct=210802&st=sb

[29] R. K. Barik, R. K. Lenka, K. R. Rao, and D. Ghose, "Performance analysis of virtual machines and containers in cloud computing," in *2016 international*

conference on computing, communication and automation (iccca). IEEE, 2016, pp. 1204–1210, accessed: May 11, 2022.

[30] D. Bernstein, "Containers and cloud: From lxc to docker to kubernetes," *IEEE Cloud Computing*, vol. 1, no. 3, pp. 81–84, 2014, accessed: May 6, 2022.

[31] V. G. da Silva, M. Kirikova, and G. Alksnis, "Containers for virtualization: An overview." *Appl. Comput. Syst.*, vol. 23, no. 1, pp. 21–27, 2018, accessed: May 6, 2022.

[32] K. Ebrahimi, G. F. Jones, and A. S. Fleischer, "A review of data center cooling technology, operating conditions and the corresponding low-grade waste heat recovery opportunities," *Renewable and Sustainable Energy Reviews*, vol. 31, pp. 622–638, 2014, accessed: May 11, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1364032113008216

[33] A. Elsayed and N. Abdelbaki, "Performance evaluation and comparison of the top market virtualization hypervisors," in *2013 8th International Conference on Computer Engineering Systems (ICCES)*, 2013, pp. 45–50, accessed: May 5, 2022.

[34] H. Fayyad-Kazan, L. Perneel, and M. Timmerman, "Benchmarking the performance of microsoft hyper-v server, vmware esxi and xen hypervisors," *Journal of Emerging Trends in Computing and Information Sciences*, vol. 4, no. 12, pp. 922–933, 2013, accessed: May 5, 2022.

[35] J. Hwang, S. Zeng, F. y. Wu, and T. Wood, "A component-based performance comparison of four hypervisors," in *2013 IFIP/IEEE International Symposium on Integrated Network Management (IM 2013)*, 2013, pp. 269–276, accessed: April 29, 2022.

[36] A. Kovács, "Comparison of different linux containers," in *2017 40th International Conference on Telecommunications and Signal Processing (TSP)*, 2017, pp. 47–51, accessed: May 11, 2022.

[37] Z. Li, M. Kihl, Q. Lu, and J. A. Andersson, "Performance overhead comparison between hypervisor and container based virtualization," in *2017 IEEE 31st International Conference on Advanced Information Networking and Applications (AINA)*, 2017, pp. 955–962, accessed: May 5, 2022.

[38] Z. Li, "Comparison between common virtualization solutions: Vmware workstation, hyper-v and docker," in *2021 IEEE 3rd International Conference on Frontiers Technology of Information and Computer (ICFTIC)*. IEEE, 2021, pp. 701–707, accessed: May 11, 2022.

[39] H. Liu, "Big data drives cloud adoption in enterprise," *IEEE Internet Computing*, vol. 17, no. 4, pp. 68–71, 2013, accessed: April 29, 2022.

[40] Y. Liu, X. Wei, J. Xiao, Z. Liu, Y. Xu, and Y. Tian, "Energy consumption and emission mitigation prediction based on data center traffic and pue for global data centers," *Global Energy Interconnection*, vol. 3, no. 3, pp. 272–282, 2020, accessed: May 11, 2022.

[41] H. Manninen, V. Jääskeläinen, and J. O. Blech, "Performance evaluation of containerization platforms for control and monitoring devices," in *2020 25th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, vol. 1, 2020, pp. 1061–1064, accessed: May 11, 2022.

[42] P. E. N, F. J. P. Mulerickal, B. Paul, and Y. Sastri, "Evaluation of docker containers based on hardware utilization," in *2015 International Conference on Control Communication Computing India (ICCC)*, 2015, pp. 697–700, accessed: May 5, 2022.

[43] K. Nanath and R. Pillai, "A model for cost-benefit analysis of cloud computing," *Journal of International Technology and Information Management*, vol. 22, no. 3, p. 6, 2013, accessed: May 11, 2022.

[44] B. Narasimhan and R. Nichols, "State of cloud applications and platforms: The cloud adopters' view," *Computer*, vol. 44, no. 3, pp. 24–28, 2011, accessed: April 29, 2022.

[45] V. Parida, *Digitalization*, 2018, p. 23–38, [ed] Editors Johan Frishammar Åsa Ericson, Accessed: May 5, 2022. [Online]. Available: http://urn.kb.se/resolve?urn=urn:nbn:se:ltu:diva-68008

[46] A. Randal, "The ideal versus the real: Revisiting the history of virtual machines and containers," *ACM Computing Surveys (CSUR)*, vol. 53, no. 1, pp. 1–31, 2020, accessed: May 11, 2022.

[47] J. Shuja, A. Gani, S. Shamshirband, R. W. Ahmad, and K. Bilal, "Sustainable cloud data centers: a survey of enabling techniques and technologies," *Renewable and Sustainable Energy Reviews*, vol. 62, pp. 195–214, 2016, accessed: May 11, 2022.

[48] W. Van Heddeghem, S. Lambert, B. Lannoo, D. Colle, M. Pickavet, and P. Demeester, "Trends in worldwide ict electricity consumption from 2007 to 2012," *Computer Communications*, vol. 50, pp. 64–76, 2014, accessed: May 11, 2022.

[49] R. L. Villars, C. W. Olofson, and M. Eastwood, "Big data: What it is and why you should care," *White paper, IDC*, vol. 14, pp. 1–14, 2011, accessed: April 29, 2022.

[50] R. Xu, G. Shi, J. Luo, Z. Zhao, and Y. Shu, "Muzi: Multi-channel zigbee networks for avoiding wifi interference," in *2011 International Conference on Internet of Things and 4th International Conference on Cyber, Physical and Social Computing*. IEEE, 2011, pp. 323–329, accessed: May 12, 2022.

Faculty of Computing, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden